

American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Project Report on Programming in Python

DHAKA WEATHER FORECAST

Semester: Fall 26-27

Group: 2		***Section: D***
SL	**Student Name**	**Student ID**
1	TANVIR HOSEN NISHAT	23-51941-2
2	MUHAMMAD TANZIMUL ISLAM	23-55693-3
3	MD AL AMIN	23-52648-2
4	KHAN SAJID AHMED	23-51685-2
5	ASHFAQ NUR	23-51714-2

1. Problem Statement, Objectives and Intended Users

1.1 Problem Statement

Dhaka has extensive hourly historical weather observations, but raw CSV tables are difficult for students and beginner analysts to inspect, compare, or maintain. Searching for a specific hour, calculating monthly summaries, identifying unusual rainfall or heat, and evaluating simple estimates otherwise require repeated manual spreadsheet work. This project provides a local desktop interface for these tasks using the supplied 2019–2024 Dhaka dataset.

A user can browse and search observations, edit a separate working copy, examine trends, generate educational moving-average estimates, and export results. Its value is practical learning and reproducible record management; it is not an operational warning system or a real-time forecasting service.

1.2 Domain and Application Type

Domain: Environment, weather-data education and information management.
Application type: Hybrid (Tkinter GUI management with CSV-based analysis and forecasting).

1.3 Objectives

- Provide searchable, editable hourly weather records with durable CSV save/load.
- Calculate descriptive statistics, trends, historical event screening and forecast accuracy metrics.
- Demonstrate modular Python programming, list/tuple/set/dictionary use, OOP, validation and an approved Tkinter GUI.

1.4 Intended Users

University students learning Python, teachers demonstrating data structures and OOP, and beginner environmental-data users who need a simple local tool to explore Dhaka's historical observations. Users interact through the GUI screens or the optional terminal interface.

1.5 Data and Privacy Note

The supplied 2019–2024 file contains non-personal environmental observations attributed in its header to NASA POWER. Edits are stored locally in a separate working CSV. The application does not request names, contact details, or other sensitive personal information. Example inputs in this report use non-sensitive test values; the original source CSV is retained.

2. Feature List and Application Workflow

2.1 Feature List

ID	Feature / Operation	Description
F-1	Add hourly observation	Validate a date and five measurements; insert a unique timestamp and save the record.
F-2	View and search records	Browse and filter observations by year, month, day, or exact timestamp.
F-3	Update existing record	Modify an existing observation and persist its new values.
F-4	Delete observation	Remove a confirmed record; do not remove the final remaining record.
F-5	Summaries and trends	Compute coverage, monthly/yearly statistics and visual trend summaries.
F-6	Forecast and evaluation	Produce 1–24 hour moving-average estimates and calculate MAE/RMSE on historical data.
F-7	Save, reload and export	Save the working CSV and export analysis or forecast results as CSV, JSON or TXT.

2.2 Application Workflow

At startup, main.py loads data/weather_records.csv when present, or reads the original source and creates a working file. The user then chooses a GUI feature, provides optional inputs, views results, saves changes, and exits. Persistent files remain available after closing the program.

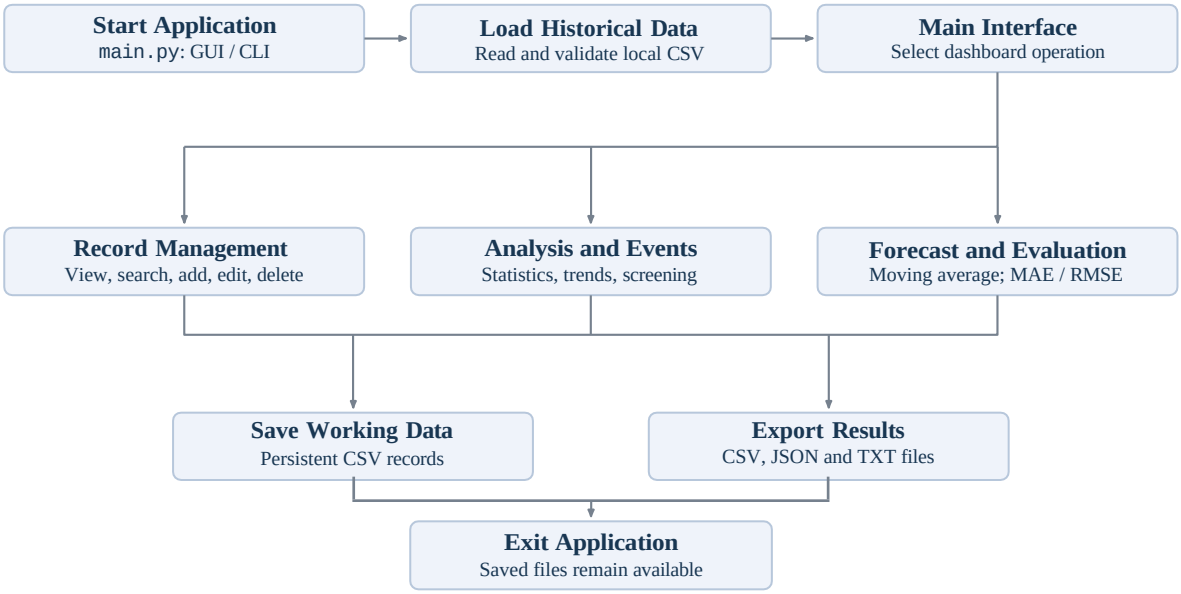


Figure 1: application workflow: startup, operations, persistence and exit.

2.3 Program Structure and File Organization

File	Purpose
main.py	Main entry point, option parsing, GUI/CLI launch.
src/weather_record.py	WeatherRecord model, timestamp parsing and value validation.
src/weather_manager.py	Records, indexes, search, CRUD, CSV save/load.
src/analytics.py, src/forecaster.py	Summaries, trend analysis, historical screening and moving aver-ages.
src/gui.py, src/gui_widgets.py	Tkinter screens, controls, tables and graphical outputs.
src/interface.py, src/reporting.py	Terminal interface and exports/reports.
data/*.csv, output/*	Historical source, working records and generated reports.

3. Python Concepts Used

For each concept, the table explains how it is used and identifies a specific source-file reference.

Concept	How it is used in this project	Function / Class / File
Variables and data types	Timestamps, readings, choices, totals and flags stored in strings, integers, floats and Booleans.	WeatherRecord; main.py
Operators	Addition, division, powers, comparisons and membership checks for means, RMSE and validation.	analytics.py; calculate_errors()
Conditional statements (if / elif / else)	Select actions, check input bounds and detect missing records or duplicates.	WeatherApp.run(); validate_number()
Loops	Read CSV rows, organize observations, pro-cess parameters and generate forecast hours.	WeatherManager.load(); forecast()
Functions	Separate parsing, filtering, CRUD, calculations, saving and report export.	parse_timestamp(); calculate_errors()

Short source-code reference: src/forecaster.py calculates MAE with arithmetic and a loop:

```
def calculate_errors(actual, predicted):
    absolute_total = 0
    squared_total  =  0
    for i in range(len(actual)):
        difference  =  predicted[i]  -  actual[i]
        absolute_total = absolute_total +
        abs(difference) squared_total = squared_total
        + difference ** 2
    return {
        "mae": absolute_total / len(actual),
```

4. Data Structure Explanation

Data Struc-ture	What it stores	Why it was chosen	Where used
List	Ordered WeatherRecord objects, forecast observations.	Easy ordered iteration and reporting.	WeatherManager.records; predictions
Tuple	Fixed parameter names, menu options and bounds.	Immutable ordered con-figuration values.	config.PARAMETERS; MENU_OPTIONS
Dictionary	Timestamp-to-record lookup and CSV key/value fields.	Fast record lookup by key.	by_timestamp; to_row()
Set	Unique imported times-tamps and available years.	Efficient uniqueness checks and duplicate prevention.	WeatherManager.load(): seen; years

Short examples from the source:

```
# List → self.records = []

# Tuple → PARAMETERS = ("T2M", "RH2M", "PS", "WS10M", "PRECTOTCORR")

# Dictionary → self.by_timestamp = {}

# Set -> seen = set()
```

5. Object-Oriented Design

5.1 Class Overview

Class Name	Purpose
WeatherRecord	Represents one validated hourly weather observation.
WeatherManager	Coordinates records, search, modification and CSV persistence.
RollingForecaster	Generates educational moving-average forecasts and evaluation results.
WeatherGUI / RecordDialog	Tkinter interface and observation editing dialog.
WeatherApp	Optional terminal menu connected to the same data-management logic.

5.2 Class Details

Class relationship: The manager contains WeatherRecord instances; the GUI use the manager, while the forecaster reads stored observations. There is no database or trained AI model.

Attributes	Methods
WeatherRecord: timestamp: datetime; five numeric measurements; record_id: str.	values(), to_row(), display().
WeatherManager: records: list; by_timestamp: dict; years: set; store_path.	load(), save(), add(), update(), delete(), select().
RollingForecaster: manager: WeatherManager.	forecast(cutoff,horizon,window), backtest(year,window).
WeatherGUI: Root window, manager, output path, view variables.	Screen navigation, record filtering, analysis and GUI lifecycle methods.
RecordDialog: Application reference, original record, input variables.	Submit user input and validate/modify records through the manager.
WeatherApp: Manager, forecaster, output path.	run(), search, add, update, delete and export operations.

5.3 Class Diagram

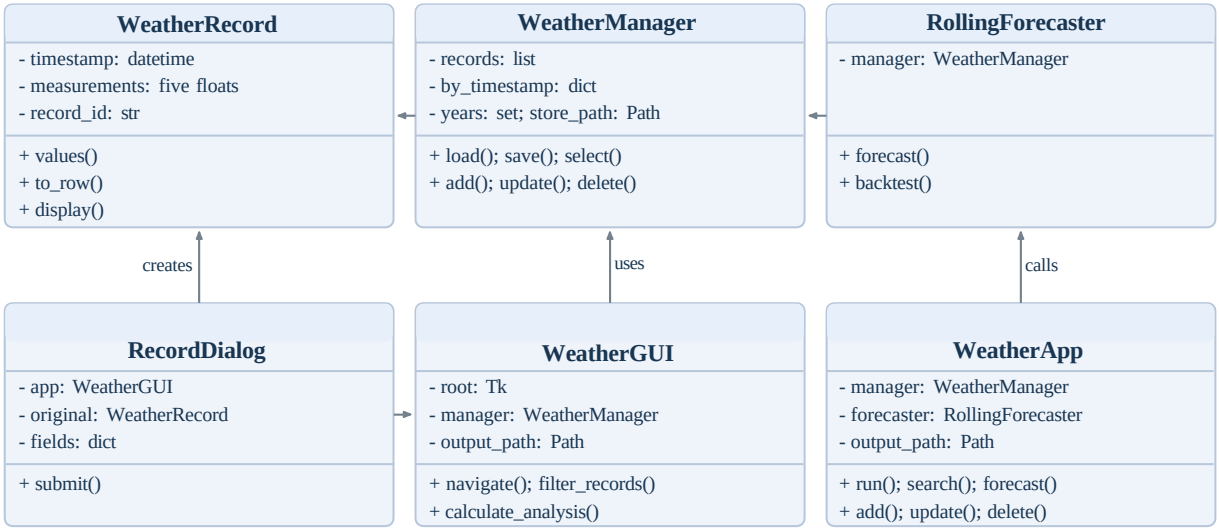


Figure 2: class diagram with attributes, methods and actual code relationships.

6. Exception Handling & Validation Rules

6.1 Exception Handling

Case	How it is handled	Function
Invalid menu or action	Show feedback and keep the menu available.	WeatherApp.run(); main.py
Non-numeric input	Catch conversion errors and show an action-able prompt.	read_integer(); RecordDialog.submit()
Out-of-range value	Reject invalid measurements and dates be-fore storage.	validate_number(); parse_timestamp()
Duplicate identifier	Check timestamps during import and add.	WeatherManager.load(); add()
Record not found	Report missing timestamp; do not alter stored data.	WeatherManager.update(); delete()
File error	Report invalid/unreadable data; do not re-place valid in-memory records on failed load.	WeatherManager.load(); write_csv_file()

6.2 Input Validation Rules

- Timestamp must be a real calendar time formatted YYYY-MM-DD HH:00; minutes must be 00. Accepted record years are 1900–2100.
- Measurement ranges: temperature −60 to 60 °C; relative humidity 0–100%; pressure 80–110 kPa; wind 0–100 m/s; rainfall 0–300 mm/hour. Values must be numeric and finite.
- An hourly timestamp must be unique. Update/delete requires an existing record, and deleting the last remaining record is prohibited.
- Bad CSV headers, missing required columns, malformed rows and files without valid observations are handled with clear errors; previous valid records are retained on a failed load.

7. Library Use and Justification

Library/module used: Tkinter, including ttk, filedialog, and messagebox.

Other standard modules: csv, json, pathlib, os, datetime, calendar, math, concurrent. futures, subprocess, and sys.

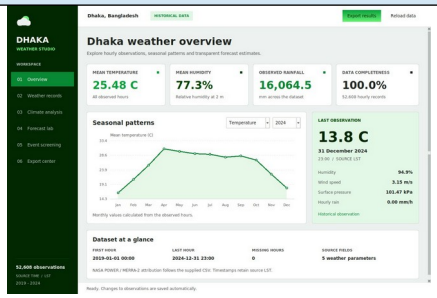
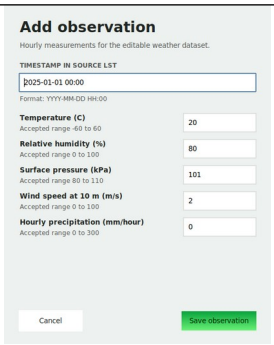
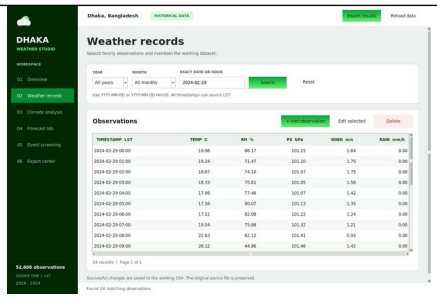
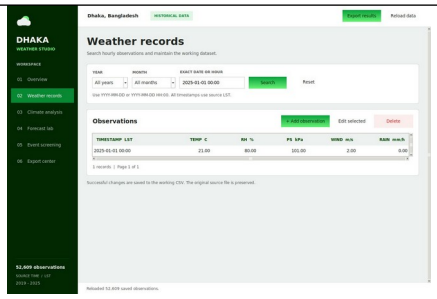
Purpose and justification: Tkinter is used for actual GUI forms, buttons, data tables, navigation, status messages and Canvas charts. It is one of the interface libraries expressly approved in the project requirements. The app does not require a third-party charting or machine-learning package to deliver these features.

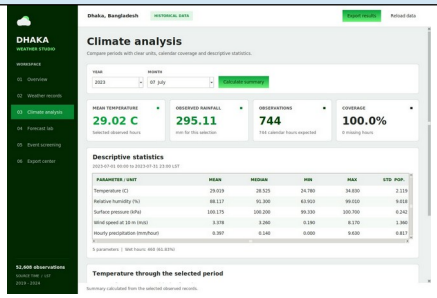
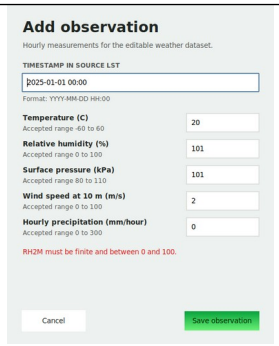
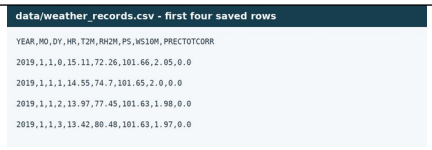
Where it is used: src/gui.py provides WeatherGUI and RecordDialog; src/gui_widgets.py contains UI widgets such as paginated tables and chart displays; main.py launches the application.

How core Python logic is still implemented by the group: CSV reading and writing, record validation, list/dictionary/set organization, add/update/delete, descriptive statistics, moving-average estimation and MAE/RMSE calculations are implemented in Python functions and classes in the source modules. Tkinter supplies interaction but does not replace that logic.

8. Screenshots and Sample Input/Output

The screenshots show the application interface, record operations, input validation, analysis, and CSV persistence.

No.	Screenshot	Description
1		**Figure 3.** Main screen: overview of the working dataset, coverage, weather metrics and visual summary.
2		**Figure 4.** Adding a record: user enters an hourly timestamp and five measurements.
3		**Figure 5.** Viewing/searching records: filtered hourly data in a paged table.
4		**Figure 6.** Record update/save result: persistence confirmation after editing.

No.	Screenshot	Description
5		**Figure 7.** Summary and climate analysis: period-based calcu-lations and visual trends.
6		**Figure 8.** Invalid input/error handling: humidity above 100% is rejected.
7		**Figure 9.** Saved data file: CSV header and sample stored weather rows.

Sample Input/Output

Sample input: To demonstrate validation, enter 2025-01-01 00:00, temperature 20, humidity 101, pressure 101, wind 2, and rain 0. This record is not added because humidity exceeds 100%. For a valid analysis example, select *July 2023*.

Sample output: The invalid humidity is rejected with the message "*RH2M must be between 0 and 100.*" For July 2023, the working project results show 744 records in 744 expected hours (100% coverage), a mean temperature of approximately 29.019 ◦C, rainfall of 295.11 mm, and 460 wet hours. The program can export results as CSV and JSON.

9. Limitations, Societal Benefit and Future Improvements

9.1 Limitations

- The application uses a finite 2019–2024 historical dataset rather than live observations. Future timestamps are educational estimates, not real-time forecasts or public warnings.
- Its rolling-average method may not capture abrupt changes or seasonal behaviour. The accuracy of results depends on the source file and data coverage.
- It is a local, single-user desktop application without a multi-user database, calibrated alert service, probabilistic uncertainty interval, or deployed online service.

9.2 Societal Benefit

Students and teachers can explore Dhaka's historical weather, reproduce calculations, learn input validation and data persistence, and examine non-sensitive environmental information. This reduces manual work when inspecting large hourly datasets and supports introductory programming education. Historical event screening is exploratory, not a replacement for official weather warnings.

9.3 Possible Future Improvements

- Add optional current meteorological data feeds with source verification and separate timestamps.
- Compare baseline forecasting methods, add out-of-sample evaluation and uncertainty representation, and extend report visualizations.
- Introduce database storage, multi-user accounts and automated regression tests if the scope later expands.

10. Conclusion

Dhaka Weather Forecasting and Trend Analysis is a Python/Tkinter application for browsing historical hourly weather observations, performing validated record addition/update/deletion, persisting local CSV data, calculating summaries and event screens, and generating basic moving-average estimates with accuracy metrics. The project uses structured functions, meaningful classes, and list, tuple, set and dictionary data structures. Core calculations remain in plain Python while Tkinter enables interaction. The implementation demonstrates the main course outcomes within an educational historical-data scope rather than claiming to be a production weather service.

Appendix A: Requirements Checklist

ID	Requirement	Done	Where demonstrated
FR-1	User interaction (menu/GUI/visual)	Yes	src/gui.py; src/interface.py; Section 8.
FR-2	At least five meaningful operations	Yes	Section 2.1, F-1–F-7; CRUD, search, analysis and export.
FR-3	Persistent data (save/load)	Yes	WeatherManager.load/save; data/weather_records.csv.
FR-4	Validation of invalid input	Yes	WeatherRecord; parse_timestamp(); Section 6.
FR-5	Analysis or calculation	Yes	src/analytics.py; src/forecaster.py.
FR-6	Structured implementation (functions and classes)	Yes	Sections 3 and 5; modular src/*.py files.
FR-7	Error recovery without crashing	Yes	Section 6; GUI validation, CLI prompts, file-error handling.

Appendix B: Submission Checklist

- ☑️ All `.py` source files, including the entry point `main.py`.
- ☑️ All sample data files, including the historical and working CSV files.
- ☑️ Complete project report.
- ☑️ Screenshots of major features.
- ☑️ Ready for live demonstration and viva.